API Automation 1 Advanced — Problem Statement

You are required to build an **end-to-end API automation test suite** for a public REST API using:

- REST Assured
- TestNG
- Jackson
- ExtentReports

The API under test (AUT) is:

- **Base URL:** https://reqres.in/api

Submission Instructions

- Submit **ONLY ONE** .java file.
- Do **NOT** create additional classes/files.
- All implementations (POJOs, tests, setup, main method) must be inside the same file.

Objective

Implement a complete API test suite that:

- Covers **CRUD operations**
- Uses **POJO-based serialization/deserialization**
- Validates **JSON schema**
- Demonstrates **request chaining (auth token)**
- Supports **data-driven testing**
- Generates **test execution reports**

What is Already Provided

You will be given a template that already includes:

- All required imports and dependencies
- Constants (BASE_URL, credentials, etc.)
- Logger setup

- JSON schema string for validation
- Extent Report setup (fully implemented)
- Pre-built POJOs:
    - UserData
    - UserSingleResponse
    - UserListResponse
- Test lifecycle hooks:
    - @BeforeSuite, @AfterSuite
    - @BeforeMethod, @AfterMethod
- Helper method: step()

CRITICAL: Do NOT modify these sections.

Tasks to Implement

TODO 1 — Create POJO Classes

Create the following inner classes using Jackson annotations:

- CreateUserRequest
- CreateUserResponse
- LoginRequest
- LoginResponse

Use:

- @JsonIgnoreProperties(ignoreUnknown = true)
- @JsonProperty(...)

TODO 2 — Configure REST Assured

Implement @BeforeClass setup:

- Use RequestSpecBuilder
- Set: Base URI

TODO 4 — Test Cases

Implement the following 8 test cases:

- **TC001 — GET Users List**
    - **Endpoint:** /users?page=1
    - **Validate:**
        - Status = 200
        - total = 12
        - total_pages = 2
        - data size = 6
        - Response time < 3 sec
- **TC002 — GET Single User**
    - **Endpoint:** /users/2
    - **Validate:**
        - Status = 200
        - JSON Schema validation
        - Deserialize response into POJO
    - **Assert:**
        - email
        - first name
        - id
- **TC003 — User Not Found**
    - **Endpoint:** /users/23
    - **Validate:**
        - Status = 404
- **TC004 — Create User (POST)**
    - **Endpoint:** /users
    - **Use:** POJO request

    - **Validate:**
        - Status = 201
        - name, job match
        - id NOT null
        - createdAt NOT null
- **TC005 — Update User (PUT)**
    - **Endpoint:** /users/2
    - **Validate:**
        - Status = 200
        - Updated name & job
        - updatedAt present
- **TC006 — Delete User**
    - **Endpoint:** /users/2
    - **Validate:**
        - Status = 204
- **TC007 — Login + Token Chaining**
    - **POST** /login
    - **Extract:** token
    - **Use token in:** Authorization: Bearer <token>
    - **GET** /users
    - **Validate:**
        - Login success
        - Token not null
        - Chained request works
- **TC008 — Data-Driven User Creation**
    - **Use** @DataProvider

    - **POST** /users for each dataset
    - **Validate:**
        - Status = 201
        - name & job match
        - id present

TODO 5 — Programmatic Test Execution

Implement the main() method:

- Create XmlSuite
- Add XmlTest
- Add class using XmlClass
- Run via TestNG
- Print report directory path

Bonus Expectations (Implicit)

- Clean code structure
- Proper assertions
- Meaningful logging
- Reusable request specification
- Correct use of serialization/deserialization

Important Rules

- Do NOT modify given code
- Do NOT create multiple files
- Do NOT hardcode unnecessary values
- Follow instructions strictly
- Ensure code compiles and runs

Evaluation Criteria

- **UI/UX:** 15%

- **CodeQuality:** 25%
- **Architecture:** 25%
- **Functionality:** 30%
- **DatabaseDesign:** 5%

```
import com.aventstack.extentreports.*;
import com.aventstack.extentreports.reporter.*;
import com.aventstack.extentreports.reporter.configuration.Theme;
import com.fasterxml.jackson.annotation.*;
import com.fasterxml.jackson.databind.ObjectMapper;
import io.restassured.RestAssured;
import io.restassured.builder.RequestSpecBuilder;
import io.restassured.filter.log.*;
import io.restassured.http.ContentType;
import io.restassured.module.jsv.JsonSchemaValidator;
import io.restassured.response.Response;
import io.restassured.specification.RequestSpecification;
import org.apache.logging.log4j.*;
import org.testng.*;
import org.testng.annotations.*;
import org.testng.xml.*;

import java.io.File;
import java.lang.reflect.Method;
import java.time.*;
import java.time.format.DateTimeFormatter;
import java.util.*;
```

```
import static io.restassured.RestAssured.given;
import static org.hamcrest.Matchers.*;

public class ApiTestSuite {

  //
================================================================================
  // // — CONSTANTS (given)
  //
================================================================================
  private static final String BASE_URL    = "https://reqres.in/api";
  private static final String LOGIN_EMAIL = "eve.holt@reqres.in";
  private static final String LOGIN_PASS  = "cityslicker";
  private static final String REPORTS_DIR = "test-reports";

  //
================================================================================
  // // — JSON SCHEMA (given – use in TC002)
  //
================================================================================
  private static final String USER_SCHEMA =
    "{\"$type\":\"object\",\"required\":[\"data\"]," +
    "\"properties\":{\"data\":{\"$type\":\"object\"," +
    "\"required\":[\"id\",\"email\",\"first_name\",\"last_name\"]," +
    "\"properties\":{" +
    "\"id\":{\"$type\":\"integer\"}," +
```

```
      "\"email\":{\"$type\":\"string\"}," +
      "\"first_name\":{\"$type\":\"string\"}," +
      "\"last_name\":{\"$type\":\"string\"} }}}}" ;

  //
================================================================================
  // // — LOGGING (given)
  //
================================================================================
  private static final Logger log = LogManager.getLogger(ApiTestSuite.class);

  //
================================================================================
  // // — EXTENT REPORTS (given)
  //
================================================================================
  private static ExtentReports extent = null;
  private static final ThreadLocal<ExtentTest> extentTest = new ThreadLocal<>();

  //
================================================================================
  // // — SHARED AUTH TOKEN (set in TC007, used in same method)
  //
================================================================================
  private static String authToken = null;

  //
================================================================================
```

```
  // // — REQUEST SPEC (set in @BeforeClass, use in all tests)
  //
================================================================================
  private static RequestSpecification requestSpec;

  //
================================================================================
  // // — PRE-WRITTEN POJOs – UserData, UserSingleResponse, UserListResponse
  //
================================================================================
  @JsonIgnoreProperties(ignoreUnknown = true)
  static class UserData {
    @JsonProperty("id")       public int id;
    @JsonProperty("email")    public String email;
    @JsonProperty("first_name") public String firstName;
    @JsonProperty("last_name")  public String lastName;
    @JsonProperty("avatar")   public String avatar;
  }

  @JsonIgnoreProperties(ignoreUnknown = true)
  static class UserSingleResponse {
    @JsonProperty("data") public UserData data;
  }

  @JsonIgnoreProperties(ignoreUnknown = true)
  static class UserListResponse {
    @JsonProperty("page")      public int page;
```

```
    @JsonProperty("per_page")    public int perPage;
    @JsonProperty("total")      public int total;
    @JsonProperty("total_pages") public int totalPages;
    @JsonProperty("data")       public List<UserData> data;
  }

  //
==========================================================================
  // // TODO 1 – POJO Classes
  //
==========================================================================

  /**
   * TODO 1a: CreateUserRequest
   * Annotate class: @JsonIgnoreProperties(ignoreUnknown = true)
   * Public fields:
   * String name -> @JsonProperty("name")
   * String job  -> @JsonProperty("job")
   * Used in: TC004 (POST /users body), TC005 (PUT /users/2 body), TC008 (DataProvider)
   */
  // static class CreateUserRequest { ... }

  /**
   * TODO 1b: CreateUserResponse
   * Annotate class: @JsonIgnoreProperties(ignoreUnknown = true)
   * Public fields:
   * String name     -> @JsonProperty("name")
```

```
 * String job      -> @JsonProperty("job")
 * String id       -> @JsonProperty("id")       NOTE: id is a String in response
 * String createdAt -> @JsonProperty("createdAt")
 * Used in: TC004 – deserialized from POST /users 201 response
 */
// static class CreateUserResponse { ... }

/**
 * TODO 1c: LoginRequest
 * Annotate class: @JsonIgnoreProperties(ignoreUnknown = true)
 * Public fields:
 * String email    -> @JsonProperty("email")
 * String password -> @JsonProperty("password")
 * Used in: TC007 – POST /login body
 */
// static class LoginRequest { ... }

/**
 * TODO 1d: LoginResponse
 * Annotate class: @JsonIgnoreProperties(ignoreUnknown = true)
 * Public fields:
 * String token -> @JsonProperty("token")
 * Used in: TC007 – deserialized from POST /login 200 response
 */
// static class LoginResponse { ... }
```

```
    //
================================================================================
    // // EXTENT REPORT – GIVEN (do not modify)
    //
================================================================================
    @BeforeSuite(alwaysRun = true)
    public void initReporting() {
        new File(REPORTS_DIR).mkdirs();
        String ts =
LocalDateTime.now().format(DateTimeFormatter.ofPattern("yyyyMMdd_HHmmss"));
        String path = REPORTS_DIR + "/ApiResponse_" + ts + ".html";
        ExtentSparkReporter spark = new ExtentSparkReporter(path);
        spark.config().setDocumentTitle("REST Assured API Report");
        spark.config().setReportName("Reqres.in - API Test Suite");
        spark.config().setTheme(Theme.DARK);
        extent = new ExtentReports();
        extent.attachReporter(spark);
        extent.setSystemInfo("Base URL", BASE_URL);
        extent.setSystemInfo("Framework", "REST Assured + TestNG");
        log.info("ExtentReports ready -> {}", path);
    }

    @AfterSuite(alwaysRun = true)
    public void flushReporting() {
        if (extent != null) extent.flush();
        log.info("ExtentReports flushed.");
```

```
    }

    @BeforeMethod(alwaysRun = true)
    public void setupTest(Method method) {
        Test ann = method.getAnnotation(Test.class);
        String desc = (ann != null && !ann.description().isEmpty()) ? ann.description() :
method.getName();
        ExtentTest test = extent.createTest(desc);
        extentTest.set(test);
        log.info("Started Test: {}", method.getName());
    }

    @AfterMethod(alwaysRun = true)
    public void reportResult(ITestResult result) {
        if (result.getStatus() == ITestResult.FAILURE) {
            log.error("X FAILED: {}", result.getName());
            extentTest.get().fail(result.getThrowable());
        } else if (result.getStatus() == ITestResult.SKIP) {
            log.warn("! SKIPPED: {}", result.getName());
            extentTest.get().skip("Skipped");
        } else {
            log.info("PASSED: {}", result.getName());
            extentTest.get().pass("Test Passed");
        }
    }
```

```
    private void step(String msg) {
        log.info(" STEP: {}", msg);
        extentTest.get().info(msg);
    }

    //
==========================================================================
    // // TODO 2 — @BeforeClass: Configure REST Assured
    //
==========================================================================

    /**
     * Annotate with @BeforeClass
     * Method name: configureRestAssured()
     * * Use RequestSpecBuilder to build requestSpec:
     * .setBaseUri(BASE_URL)
     * .setContentType(ContentType.JSON)
     * .addFilter(new RequestLoggingFilter())   <- logs every request
     * .addFilter(new ResponseLoggingFilter())  <- logs every response
     * .build()
     * * Store result in static field 'requestSpec'
     * Log: "RequestSpec configured -> BASE_URL"
     */
    // TODO 2: write @BeforeClass here
```

```
    //
================================================================================
    // // TODO 3 — @DataProvider
    //
================================================================================

    /**
     * Annotate with: @DataProvider(name = "userCreationData")
     * Returns Object[][] with exactly 3 rows.
     * Each row: { name, job }
     * * Row 1: { "morpheus", "leader" }
     * Row 2: { "neo",      "the one" }
     * Row 3: { "trinity",  "operator" }
     */
    // TODO 3: write @DataProvider here



    //
================================================================================
    // // TODO 4 — TEST CASES
    //
================================================================================

    /**
     * TC001 — GET UserList
     * @Test: groups={"smoke","get"}, description="TC001: GET /users - status 200, verify
total, data size, response time", priority=1
```

```
    * * given(requestSpec)
    * .queryParam("page", 1)
    * .when()
    * .get("/users")
    * .then()
    * .statusCode(200)
    * .body("total", equalTo(12))
    * .body("total_pages", equalTo(2))
    * .body("data", hasSize(6))
    * .body("data[0].id", notNullValue())
    */
}
//
============================================================================
   // // TODO 4 — TEST CASES (Continued)
   //
============================================================================

   /**
    * TC001 — GET UserList
    * @Test: groups={"smoke","get"}, description="TC001: GET /users - status 200, verify
total, data size, response time", priority=1
    * * given(requestSpec)
    * .queryParam("page", 1)
    * .when()
    * .get("/users")
    * .then()
```

```
    * .statusCode(200)
    * .body("total",     equalTo(12))
    * .body("total_pages", equalTo(2))
    * .body("data",       hasSize(6))
    * .body("data[0].id",  notNullValue())
    * .time(lessThan(3000L))  <- response time assertion (milliseconds)
    */
   // // TODO 4a: TC001_GetUserList



   /**
    * TC002 – GET Single User: Jackson deserialization + JSON schema validation
    * @Test: groups={"smoke","get"}, description="TC002: GET /users/2 - Jackson POJO + schema validation", priority=2
    * * UserSingleResponse resp = given(requestSpec)
    * .when().get("/users/2")
    * .then()
    * .statusCode(200)
    * .body(JsonSchemaValidator.matchesJsonSchema(USER_SCHEMA))  <- schema validation
    * .extract().as(UserSingleResponse.class);              <- Jackson deserialization
    * * Assert: resp.data != null
    * Assert: resp.data.email == "janet.weaver@reqres.in"
    * Assert: resp.data.firstName == "Janet"
    * Assert: resp.data.id == 2
    */
   // // TODO 4b: TC002_GetSingleUser
```

```
   /**
    * TC003 – User Not Found (Negative Test)
    * @Test: groups={"smoke","negative"}, description="TC003: GET /users/23 - expect 404", priority=3
    * * given(requestSpec).when().get("/users/23").then().statusCode(404)
    */
   // // TODO 4c: TC003_UserNotFound



   /**
    * TC004 – POST Create User
    * @Test: groups={"smoke","post"}, description="TC004: POST /users - 201, verify name/job/id/createdAt", priority=4
    * * 1. Create CreateUserRequest: name="morpheus", job="leader"
    * 2. POST and deserialize to CreateUserResponse:
    * given(requestSpec).body(req).when().post("/users")
    * .then().statusCode(201).extract().as(CreateUserResponse.class)
    * 3. Assert: resp.name == "morpheus"
    * 4. Assert: resp.job  == "leader"
    * 5. Assert: resp.id       is not null
    * 6. Assert: resp.createdAt is not null
    * * TIP: To demonstrate manual ObjectMapper (Jackson) parsing, include as a comment:
    * // String json = given(requestSpec).body(req).when().post("/users").asString();
    * // CreateUserResponse resp = new ObjectMapper().readValue(json, CreateUserResponse.class);
```

```
    */
    // // TODO 4d: TC004_CreateUser



    /**
     * TC005 – PUT Update User
     * @Test: groups={"regression","put"}, description="TC005: PUT /users/2 - 200, verify name/job/updatedAt", priority=5
     * * 1. Create CreateUserRequest: name="morpheus", job="zion resident"
     * 2. given(requestSpec).body(req).when().put("/users/2")
     * .then()
     * .statusCode(200)
     * .body("name",     equalTo("morpheus"))
     * .body("job",      equalTo("zion resident"))
     * .body("updatedAt", notNullValue())
     */
    // // TODO 4e: TC005_UpdateUser



    /**
     * TC006 – DELETE User
     * @Test: groups={"regression","delete"}, description="TC006: DELETE /users/2 - expect 204 No Content", priority=6
     * * given(requestSpec).when().delete("/users/2").then().statusCode(204)
     */
    // // TODO 4f: TC006_DeleteUser
```

```
/**
 * TC007 – Login + Bearer Token (Request Chaining)
 * @Test: groups={"regression","auth"}, description="TC007: POST /login, extract token, chain as Bearer in GET /users", priority=7
 * * STEP 1 - Login:
 * LoginRequest loginReq = new LoginRequest();
 * loginReq.email = LOGIN_EMAIL; loginReq.password = LOGIN_PASS;
 * * LoginResponse loginResp = given(requestSpec).body(loginReq)
 * .when().post("/login")
 * .then().statusCode(200).body("token", notNullValue())
 * .extract().as(LoginResponse.class);
 * * authToken = loginResp.token;
 * Assert.assertNotNull(authToken);
 * * STEP 2 - Chained request using token:
 * given(requestSpec)
 * .header("Authorization", "Bearer " + authToken)
 * .when().get("/users")
 * .then().statusCode(200).body("data", hasSize(greaterThan(0)))
 */
// // TODO 4g: TC007_LoginAndBearerToken



/**
 * TC008 – Data-Driven POST via @DataProvider
 * @Test: groups={"regression","data-driven"}, description="TC008: Data-driven POST /users - 3 users from @DataProvider", dataProvider="userCreationData", priority=8
```

```
    * signature: TC008_DataDrivenCreate(String name, String job)
    * * 1. step("Creating user: name=" + name + ", job=" + job)
    * 2. Build CreateUserRequest with name and job
    * 3. given(requestSpec).body(req).when().post("/users")
    * .then()
    * .statusCode(201)
    * .body("name", equalTo(name))
    * .body("job",  equalTo(job))
    * .body("id",   notNullValue())
    */
   // // TODO 4h: TC008_DataDrivenCreate



   //
==========================================================================
   // // TODO 5 — main(): Programmatic TestNG Run
   //
==========================================================================


   /**
    * 1. XmlSuite -> name="Reqres API Suite", parallel=NONE
    * 2. XmlTest  -> name="Full API Regression", add ApiTestSuite.class via XmlClass
    * Comment showing smoke-only run:
    * // xmlTest.setIncludedGroups(Collections.singletonList("smoke"));
    * 3. TestNG runner -> setXmlSuites -> run()
    * 4. Print "Reports -> test-reports/"
    */
```

```
    public static void main(String[] args) {
        // // TODO 5
    }
}
```